

Morse Code Based ESP32 Communication with LLM Integration

S. V. Ashok Sainaadh^{1*}, M. Neil Kumar¹,
B. Sai Sundhar Reddy¹, Mithun Kumar Kar¹

^{1*}Amrita School of Artificial Intelligence, Coimbatore, Amrita Vishwa
Vidyapeetham, Coimbatore, 641112, India.

Abstract

Morse code is a symbolic communication system that encodes letters, numbers, and characters using short and long signals (dots and dashes). Originally developed for telegraphy, it continues to serve as an efficient input method in assistive technologies, especially for individuals with speech or motor impairments due to its minimal physical effort requirement. This paper presents an assistive communication system that integrates Morse code input with ESP32 microcontrollers and Large Language Models (LLMs) to enable intelligent, natural interaction. A capacitive touch sensor connected to the ESP32 captures Morse code signals, which are decoded into text and transmitted to the Raspberry Pi via a USB serial connection. To improve reliability, an intermediate error-correction stage is applied before final language response generation. This stage reduces dot-dash timing ambiguity and partial-token decoding errors using user-calibrated timing thresholds and context-aware text correction. The Raspberry Pi then forwards the corrected text to an LLM for contextual processing and generates a meaningful response, which is sent back to the ESP32 for real-time display on an OLED or LCD module and to phone. Morse code enables low-bandwidth, reliable communication in environments where speech, typing, or high-speed data transfer is difficult or impossible. This system is especially useful in assistive communication for individuals with speech disabilities, paralysis, ALS, or motor impairments, where even minimal finger or touch movement can generate meaningful communication. Experimental evaluation over 1000 trials shows that the complete two-stage Morse-to-LLM pipeline achieves above 99% end-to-end communication accuracy.

Keywords: Morse Code, ESP32, Large Language Models, Health Monitoring, Assistive Technology, Human-Computer Interaction

1 Introduction

The dot dash signaling system, one of the earliest data transmission methods, converts textual characters into sequences of dots and dashes arranged according to precise interval protocols that separate signals into symbols, letters, and words [27, 1, 19]. This time-sensitive dual-state signaling system remains well-suited for transmitting data over low-bandwidth or noisy channels, where power limitations or signal integrity are critical [12, 18]. Its simplicity allows cognitively straightforward message encoding by individuals with neuromotor impairments or speech disabilities, making it relevant beyond historical telegraphy into modern assistive communication devices [3, 10].

The integration of a dot-dash signaling system in assistive technologies has provided significant independence to individuals with verbal communication or motor disabilities, facilitating message transmission through alternative channels when conventional methods are insufficient [5, 1, 7]. Its applications span assistive control interfaces, emergency distress signaling, and advanced sensory and wireless communication devices designed for enhanced usability [2, 6]. The system’s compatibility with physical limitations through button-activated or touch-based dot-dash input broadens its accessibility to a wide spectrum of users with varying disabilities [17, 10].

The ESP32 microcontroller [23], has emerged as a preferred platform for developing Morse-based assistive devices. It offers substantial processing capability, onboard Wi-Fi and Bluetooth connectivity, and power-efficient operation for handling data transmissions [13, 10, 7]. Its compact hardware architecture supports capacitive touch sensing, real-time Morse code decoding, and interfacing of biosignal sensors to monitor physiological indicators such as heart rate and blood oxygen saturation [28, 18]. Its cost-effectiveness and flexibility make the ESP32 well-suited for rapid prototyping and deployment of accessible communication systems [2].

In the current technological climate, breakthroughs in Large Language Models (LLMs) [24] have made possible context-sensitive and probabilistic language understanding capabilities, which can restructure brief Morse input into expressive and logically structured information transmission [29, 14]. Integrating LLMs with Morse code input promotes noise compensation, user intention identification, and augmented conversational dialogue, substantially minimizing limitations in classical Morse-based communication devices [14]. This convergence of onboard sensing and artificial intelligence based technologies strengthens user satisfaction and broadens communication functionalities [4].

While dot-dash based systems provide fundamental communication channels, they face considerable constraints, including slow manual signal rate and considerable skill acquisition requirements, which impact the performance of the design [1, 19, 10]. Additionally, irregularity in signal timing patterns and signal disturbance creates barriers for consistent signal interpretation [19, 1]. Many current systems lack smart, adaptive responses or cohesive interfacing with hand-movement interpretation and biosignal monitoring, thus restricting human-like communication and versatility [6]. Stable, multi-modal accessibility systems using Morse code remain a high-impact research area. Fig 1 represents a conventional scenario in which the user is incapable of interacting efficiently due to communication and motor disorders, limiting interaction with assistants and access to remote information. However, with the designed

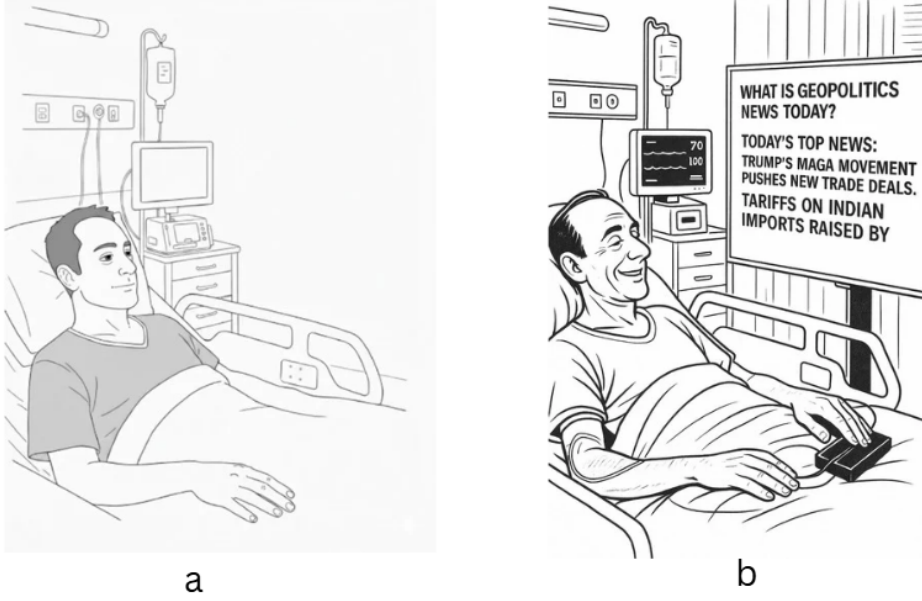


Fig. 1 Conceptual illustration of user communication capability in assistive settings. (a) Conventional scenario (b) Proposed assistive communication scenario

communication architecture, gesture-based dot-dash input enables interaction with a cloud-hosted AI language model, allowing the user to transmit meaning, access live data, and communicate efficiently with caregivers and digital systems, thereby enhancing self-governance, accessibility, and overall quality of life.

Incorporating biosignal monitoring into Morse-driven communication platforms enables ongoing vital signs tracking, distress signaling, and remote health consultation, supporting telehealth and long-term elder support [13, 5, 3]. Onboard sensing modules recording SpO₂, heart rate, and temperature generate essential medical data conveyed through Morse or cable-free communication protocols, while generative AI reports provide user-friendly explanations and health advisories [13, 17, 7]. This synergy enables improved self-sufficiency and well-being for high-risk populations.

Despite these positive aspects, there remains an evident gap in incorporating deterministic Morse decoding, generative AI-supported coherent communication, and touch-triggered biosignal monitoring in a portable, power-optimized embedded architecture [19, 1, 4, 7]. Existing solutions are often limited in functionality, remotely hosted, or lack sensor fusion capabilities necessary for applied adaptive systems. The proposed work overcomes these gaps by integrating ESP32-based real-time Morse decoding, a Raspberry Pi communication bridge facilitating local or remote generative model APIs, and a biomedical sensing unit activated via intuitive tactile gestures. This advanced multimodal framework strengthens system reach, secure data handling, and articulation capability in rehabilitative communication technologies.

To improve reliability, the proposed system includes a lightweight error-correction algorithm between Morse decoding and final LLM response generation. In the first step, decoding noise is reduced using a user-calibrated timing threshold and simple signal filtering, which lowers dot-dash misclassification from inconsistent touch duration. In the second step, a correction module refines partially decoded or ambiguous text before semantic response generation. Only uncertain cases are sent to the deeper response stage, so error propagation is minimized while computation remains efficient. This staged correction design is a key reason the complete system achieves above 99% end-to-end communication accuracy over 1000 trials.

Morse code remains underutilized in modern adaptive communication systems, and most existing systems fail to incorporate it with LLMs for spontaneous, context-aware interaction. Reliable code translation on power-constrained hardware like the ESP32 microcontroller is challenging due to signal timing variations and channel noise, and available systems are commonly function-specific without integrated messaging features. Additionally, most Morse-based interfaces fail to include natural touch-based controls, diminishing accessibility for individuals with motor impairments.

The introduced adaptive communication platform validates the functional capability of combining a dot-dash interaction mechanism with large language models on a consolidated microcontroller-based platform. By assembling capacitive touch Morse signaling, touch-gesture regulation, and AI-based language processing, the system allows robust and context-sensitive interaction using a unified, affordable embedded device. The presented framework is engineered to be handheld, energy-optimized, and practical for applied inclusive and telecommunication contexts.

The main contributions of this work are summarized as follows:

1. The proposed system introduces a structured two-stage framework that separates text correction (Stage-1) from full LLM response generation (Stage-2). This modular design improves interpretability and control over the communication process.
2. A capacitive touch-driven Morse interface is designed for character-level, word-level, and sentence-level input, and maps decoded text to context-aware LLM prompts for natural and intent-preserving communication.
3. An uncertainty-aware two-stage error-correction algorithm is introduced: Stage-1 performs deterministic rule/domain correction, while Stage-2 applies semantic LLM correction only for uncertain outputs. This improves decoding robustness, reduces unresolved tokens, and lowers failed-query cases.
4. The proposed framework follows a modular assistive communication architecture with clearly separated layers for touch-based signal acquisition, Morse decoding, staged error correction, and LLM response generation. This layered design simplifies integration, improves maintainability, and enables straightforward extension to future assistive interaction modules.

The rest of this paper will be arranged in the following order: In Section 2, there will be a discussion on the related research on assistive communication systems, Morse code interfaces, and embedded AI-driven interaction frameworks. In Section 3, there will be a description of the overall system methodology and design. In Section 4,

there will be a discussion on the hardware implementation and detailed circuit design. In Section 5, there will be a description of the software architecture and the overall workflow process. In Section 6, there will be a discussion on the experimental results and how the system performs. Finally, in Section 7, there will be a discussion on user experience, limitations, and ethical issues, and then the conclusion will follow.

2 Literature Review

Many authors have proposed different technologies for implementing Morse code-based reliable communication systems that can interpret assistive medical integration, context adaptation, and predictive language modeling. Walker et al. [1] studied the haptic Morse code interface, where two hands are employed to present location-based stimuli, reducing the decoding mistakes for users with significant impairments. The researchers mapped the dots and dashes to each hand, reducing the mistakes dramatically compared to the single-hand approach. The study also shows the benefits to perception and temporal judgments in decoding the code. However, the system is limited to spatial information and touch feedback. It lacks language models for expressive output, gestures, or multimodal functionality. Additionally, it lacks biomedical sensors or the ability to generate intelligent, context-based reply mechanisms. Pravin Kumar et al. [2] developed a Raspberry Pi-based system that employs multiple Morse code buttons with text prediction capabilities and speech recognition via Bluetooth connectivity for partially paralyzed individuals. The focus is on cost-effective and flexible communication solutions with basic vital signal monitoring. In terms of actual use cases, the model seems viable. However, it is based on Morse code, manual settings, and a rule-based approach. There is no use of NLP or large language models, gestures, or biomedical data. Silva et al. [3] built a Morse code translator on Arduino for education, binary coding, and visible light communication. Their approach validates Morse in STEM skill-building and low-bandwidth communication. Its flexible encoding logic is useful for demonstration and prototyping. Limitations arise from its pedagogical focus, simple error handling, and single-mode use. Yang et al. [4] proposed “UniTouch,” a deep-learning model that aligns tactile, vision, and language modalities for zero-shot retrieval. Their system advances multimodal learning for touch-driven robotics and material identification. The study excels in unified representations but restricts itself to tactile and image data, not Morse timing or multi-sensor biosignals. Yang et al. [5] developed a wireless Morse code-controlled device for disabled persons. The study demonstrates the increasing independence of disabled persons through RF communication. The adaptive mechanism of the system is useful but is strictly based on Morse code and RF. Gaura et al. [6] created a microcontroller Morse interface for the visually impaired and mute, supporting tactile entry, screen feedback, and voice playback. It emphasizes two-way messaging and affordability for users with sensory deficits. However, it has slow input/output speed, no biomedical loop, and limited adaptability to user context. Narayan et al. [7] introduced a Morse generator using a microcontroller and keypad for composing Morse symbols with audio/visual/tactile outputs. It aids Morse code practice and basic alternative communication. Prathap et al. [8] presented a low-cost switch-based Morse text-to-speech device for partially paralyzed

persons using audio-visual feedback. The core advantage is utility at minimal cost and input complexity. However, Morse code fluency is required, the language is basic, and no adaptive, predictive, or health features are present. Ghosh et al. [9] described a power-efficient, high-speed digital comparator for embedded event detection using Morse code, with applications in sensor-driven platforms. Their design improves hardware efficiency for core signal processing tasks. Miskon et al. [10] proposed a Morse code ON-OFF signal processing unit with an ATmega328P and Bluetooth for disabled user communication. It features switch-driven sentence entry and LCD/buzzer feedback. There are several obstacles with the model: monomodal input, limited Bluetooth range, no predictive NLP or health-related applications, and lack of critical platform features such as user error corrections, cloud-based context awareness, and multi-modal triggers, which are critical components of our design. Ramesh et al. [11] implemented a system that enables blind users to interact with their surroundings through a braille or speech-based interface. The major contribution of their system is the generation of accessible visual content. Chand et al. [12] analyzed Morse for high-speed encryption/decryption, testing efficiency against traditional ciphers. Efficient computation is proven for encoded transmission. However, this use case is not optimized for communication or assistive purposes, lacks health or predictive features, and has limited utility for user messaging. Our design pivotally blends privacy, adaptive health reporting, and intelligent, personalized dialog. Chitra et al. [13] designed a Morse code-based system for ESP32 that offers SpO2, heart rate, and temperature sensing with text-to-speech functionality. Its advantages include biomedical sensing and remote reporting. However, it requires manual Morse code input, does not use NLP for dialogue systems, and does not offer context-based or LLM-based health reports. Simoni et al. [14] developed Morse, which is a chatbot for cybersecurity that utilizes Retrieval-Augmented Generation (RAG) along with a large language model. It can generate answers dynamically with the aid of reinforcement learning-based retrieval. The system excels in the provision of relevant security questions and answers as well as in its updatability. However, Morse does not support physical interfacing, Morse acquisition, or health-related data. Mohan et al. [15] discussed a basic system based on Morse code, which is affordable and accessible for users with disabilities. The interface provides foundational accessibility but is limited by slow input, lack of language expansion, and absence of health integration or context awareness. Alex et al. [16] advanced Morse decoding using convolutional neural networks to improve resilience to noise and signal variation. While this increases symbol classification accuracy, the focus is on backend signal processing; it omits assistive application, user context, or intent prediction. No biomedical or LLM-powered dialog capabilities are included, as delivered by our platform. Yang et al. [17] further validated RF Morse code control for environmental devices in a severely disabled cohort. User independence improved, but speed, error correction, automation, and health reporting were minimal. This approach lacks adaptive feedback, multi-modal input, and predictive dialog, all of which are central to our integrated LLM-ESP32 project. Yin et al. [18] developed the C-Morse method, which uses a combination of Wi-Fi and Zigbee technologies with timing modulation to send and receive Morse code. This method increases the rate and compatibility of the system but is not centered around the user or their health. It does not include

multimodal triggers, adaptive dialog, or health function scaffolding. Yang et al. [19] developed a Morse recognizer using LMS algorithms for users with hand coordination deficits. Improved accuracy and personalized recognition highlight the system’s benefit. However, it does not involve health monitoring, intelligent response generation, touch-based sensors, or LLM/NLP support, which are advanced features of our project. Suresh et al. [20] developed a system that provides a two-way interface using the Morse code method and includes hardware switches and messaging capabilities, making it accessible to people with multiple disabilities. This system has a good workflow and hardware design, making it accessible and useful. However, the system does not provide health data fusion, adaptive and predictive NLP/LLM capabilities, or gesture sensor capabilities, which our system addresses and provides solutions for.

3 System Methodology

The system that we are proposing is based on a modular and layered approach that integrates simple hardware input with rich cloud-based language processing. The proposed system is structured as a four-stage pipeline that includes input, decoding, interaction with LLM, and output. The modular design of this proposed system allows each of its parts to be fine-tuned individually without affecting other parts. Fig 2 decorates the entire workflow of Morse to LLM. The user taps on the sensor, which captures touch input and translates it into Morse code by the input module. The ESP32 then decodes it to text. The Raspberry Pi module forwards this text to the above LLM (SONAR PRO[21, 22]) through an API by the Raspberry Pi. The LLM generates text and sends it back to the user device through the ESP32, which is then displayed on the screen.

3.1 Input Acquisition

The first stage of the system captures user input in the form of Morse code signals. A capacitive touch sensor serves as the primary user interface, selected for its high sensitivity and ease of integration with the ESP32 microcontroller. The ESP32 continuously monitors the input signal and records the duration of each tap, distinguishing between:

1. Dot (·): Short press below a predefined threshold.
2. Dash (–): Longer press above the threshold.
3. Pause: Absence of signal, representing space between letters or words.

Fig. 3 represents Morse code encoding scheme for alphanumeric characters, serving as the symbolic interface between gesture-based input detection and textual query generation in the proposed assistive communication system.

This simple yet robust input mechanism makes the system accessible to users with limited motor functions. The threshold is empirically chosen as 500 ms. Fig 4 represents the hardware connection between the ESP32 microcontroller and the capacitive touch sensors used for Morse code input. Each sensor module is wired to the ESP32 through its VDD, GND, SO, and STG pins, ensuring stable power and reliable signal output. The SO (signal out) pins are connected to appropriate GPIO pins on the ESP32 to

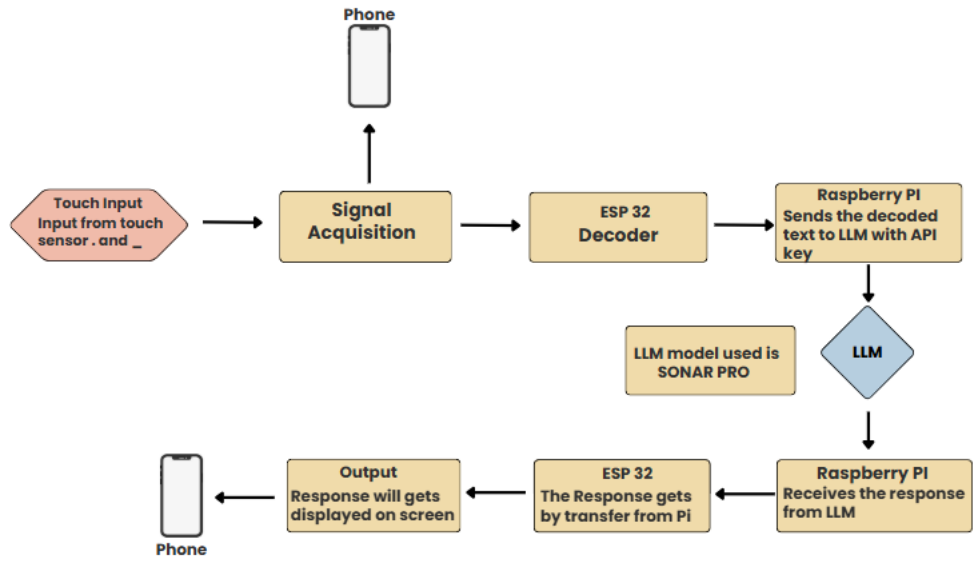


Fig. 2 The above block diagram explains how the data is being flow and then decode by LLM and then sent through micro controller and it gets displayed on screen to a user

The Morse Code

A	● —	T	— ●
B	— ● ● ●	U	● ● —
C	— ● — ●	V	● ● ● —
D	— ● ●	W	● — —
E	●	X	— ● ● —
F	● ● — ●	Y	— ● ● —
G	— — ●	Z	— — ● ●
H	● ● ● ●		
I	● ●	0	— — — — —
J	● — — —	1	● — — — —
K	— ● —	2	● ● — — —
L	● — ● ●	3	● ● ● — —
M	— —	4	● ● ● ● —
N	— ●	5	● ● ● ● ●
O	— — —	6	— ● ● ● ●
P	● — — ●	7	— — ● ● ● ●
Q	— — ● —	8	— — — ● ● ●
R	● — ●	9	— — — — ●
S	● ● ●		

Fig. 3 Morse code encoding scheme for alphanumeric characters, serving as the symbolic interface between gesture-based input detection and textual query generation in the proposed assistive communication system.

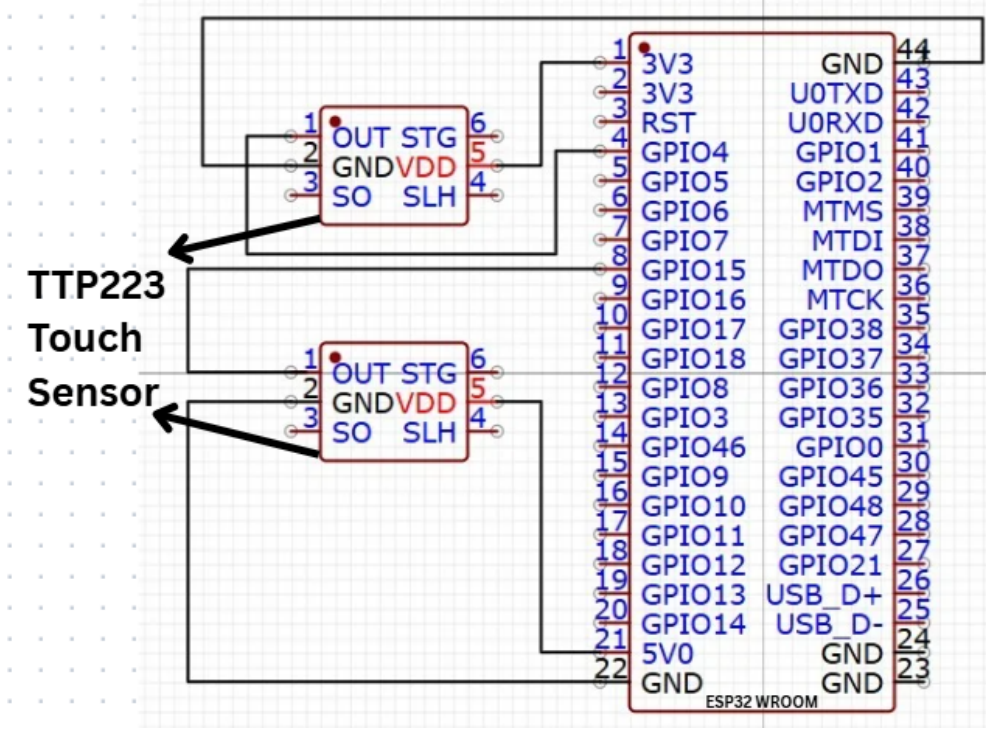


Fig. 4 This image explains about circuit connections with esp-32 micro controller and touch sensors

capture touch events accurately. The wiring layout ensures minimal noise interference and proper grounding, which is essential for consistent Morse dot-dash detection. Overall, the figure illustrates a clean and robust interface between the touch sensors and the ESP32 for real-time input processing.

3.1.1 Time and Touch Signal Representation

Let time be modeled as a continuous variable:

$$t \in \mathbb{R}^+ \quad (1)$$

The system uses two independent capacitive touch sensors:

- s_1 : Dot/Dash input sensor
- s_2 : Secondary input sensor

Each sensor produces a binary-valued signal:

$$s_i(t) \in \{0, 1\}, \quad i \in \{1, 2\} \quad (2)$$

where $s_i(t) = 1$ denotes physical contact and $s_i(t) = 0$ denotes no contact.

3.1.2 Touch Event and Duration Estimation

A touch event begins at time t_s and ends at time t_e when the signal transitions from high to low:

$$s_i(t_s) = 1, \quad s_i(t_e) = 0 \quad (3)$$

The duration of the touch event is defined as:

$$\Delta t_i = t_e - t_s \quad (4)$$

This duration is measured using the ESP32 internal clock.

3.1.3 Temporal Threshold Definition

A fixed temporal threshold τ is defined:

$$\tau = 500 \text{ ms} \quad (5)$$

The threshold is used exclusively to distinguish between Morse symbols.

3.1.4 Morse Symbol Generation Logic

Touch duration is mapped to Morse symbols using the following rule:

$$m(\Delta t_i) = \begin{cases} . & \text{if } \Delta t_i < \tau \\ - & \text{if } \Delta t_i \geq \tau \end{cases} \quad (6)$$

Interpretation:

- Short press $\rightarrow$ DOT
- Long press $\rightarrow$ DASH

3.1.5 Discrete Morse Sequence Construction

Each generated Morse symbol is denoted as:

$$m_k \in \{., -\} \quad (7)$$

A Morse character is represented as an ordered discrete sequence:

$$M^{(j)} = (m_1^{(j)}, m_2^{(j)}, \dots, m_{n_j}^{(j)}) \quad (8)$$

where j indexes the character and n_j denotes the number of symbols generated consecutively.

3.2 Decoding Module

Once the raw input pulses are detected, they are processed within the ESP32 firmware [23]. A **timing analysis algorithm** compares the recorded pulse lengths to preset thresholds, classifying them as dots, dashes, or pauses. A Morse translation table is realized to map signal sequences into alphanumeric symbols. The implementation utilized intra-character spacing to separate between separate letters, lexical spacing to clearly separate words, and message completion markers such as a extended interval or a unique signal sequence to mark message conclusion. The interpreted text is retained in onboard memory prior to data transfer. This modular decoding process ensures reliable translation even under varying input speeds.

3.2.1 Deterministic Morse Decoding

Let Σ denote the alphanumeric output space:

$$\Sigma = \{A, \dots, Z, 0, \dots, 9\} \quad (9)$$

Decoding is performed using a deterministic dictionary mapping:

$$\mathcal{D} : \{., -\}^* \rightarrow \Sigma \quad (10)$$

The decoded character is given by:

$$c_j = \begin{cases} \mathcal{D}(M^{(j)}) & \text{if mapping exists} \\ ? & \text{otherwise} \end{cases} \quad (11)$$

To provide a rigorous and unambiguous description of the proposed Morse-to-LLM pipeline, the following subsection presents a formal mathematical modeling of the touch-based signal acquisition, Morse symbol generation, deterministic decoding, and probabilistic LLM interaction

3.3 Error Correction and Uncertainty Modeling

In practical touch-based Morse acquisition, errors may arise due to signal noise, timing variability, and user inconsistencies. These errors manifest not only as invalid symbol mappings but also as linguistic inconsistencies such as misspelled words, misplaced characters, or incomplete tokens. To address this, we introduce a hybrid uncertainty-aware error correction framework that combines deterministic decoding with a software-level linguistic validation layer.

3.3.1 Uncertainty Representation

Let the decoded character sequence be:

$$Q = c_1 c_2 \dots c_K \quad (12)$$

From deterministic decoding:

$$c_j = \begin{cases} D(M^{(j)}) & \text{if mapping exists} \\ ? & \text{otherwise} \end{cases} \quad (13)$$

We define a character-level uncertainty indicator:

$$u_j = \begin{cases} 1 & \text{if } c_j = ? \\ 0 & \text{otherwise} \end{cases} \quad (14)$$

Thus, a preliminary uncertainty score is:

$$U_{\text{sym}}(Q) = \sum_{j=1}^K u_j \quad (15)$$

However, symbol-level uncertainty alone is insufficient. Therefore, the system incorporates a linguistic validation function implemented in the Python interface, which evaluates:

- Character-level inconsistencies,
- Word-level misspellings,
- Token fragmentation or misplaced characters,
- Presence of invalid or ambiguous symbols.

The overall uncertainty is defined as:

$$U(Q) = U_{\text{sym}}(Q) + f_{\text{lang}}(Q) \quad (16)$$

where $f_{\text{lang}}(\cdot)$ quantifies linguistic inconsistency.

3.3.2 LLM-Based Correction Function

To recover semantically meaningful text from imperfect input, we define a correction operator:

$$\tilde{Q} = \mathcal{C}(Q) \quad (17)$$

where:

- Q is the raw decoded query,
- $\tilde{Q}$ is the corrected query,
- $\mathcal{C}(\cdot)$ represents the LLM-based correction function.

This function leverages contextual language priors to resolve spelling errors, reconstruct fragmented tokens, and infer missing content.

3.3.3 Hybrid Decision Mechanism

A key contribution of this work is the integration of uncertainty estimation with user-controlled operation. Instead of enforcing a fully automatic decision, the system provides guidance based on uncertainty while allowing user intervention.

A threshold-based condition is defined as:

$$U(Q) \geq \theta \quad (18)$$

where θ is a predefined uncertainty threshold.

Based on this, the system suggests an operational mode; however, the final selection is controlled by the user:

$$\text{Mode} \in \{\text{Correction Only}, \text{Full Response}\} \quad (19)$$

- In **Correction Only Mode**, the system outputs $\tilde{Q}$ for user verification.
- In **Full Response Mode**, the corrected query is forwarded for semantic response generation.

This hybrid strategy combines automated uncertainty estimation with human-in-the-loop decision-making, improving both robustness and usability.

3.3.4 System-Level Interpretation

The proposed framework transforms the Morse-to-text pipeline into an uncertainty-aware and user-adaptive system. By integrating symbol-level decoding, linguistic validation, and controlled LLM interaction, the system mitigates error propagation while maintaining flexibility in real-time communication scenarios.

3.4 LLM Interaction

Sonar Pro [21, 22] is an advanced large language model developed by Perplexity[21, 22], designed for high-accuracy, search-grounded reasoning. It supports a very large context window, allowing it to process long Morse-decoded messages and generate context-rich responses. Built on a fine-tuned version of Llama 3.3, the model delivers fast inference speeds suitable for real-time communication systems. Its ability to provide reliable, citation-backed outputs makes it ideal for assistive and healthcare applications. Overall, Sonar Pro[21, 22] offers a balanced combination of speed, accuracy, and cost-efficiency, making it well-suited as LLM communication module.

3.4.1 Text Query Formation

Decoded characters are concatenated sequentially to form a raw text query:

$$Q = c_1 c_2 \dots c_K \quad (20)$$

where K is the total number of decoded Morse characters.

To account for decoding errors and linguistic inconsistencies, a correction stage is introduced:

$$\tilde{Q} = \mathcal{C}(Q) \quad (21)$$

where $\tilde{Q}$ represents the corrected query obtained via the LLM-based correction function.

3.4.2 Two-Stage Large Language Model Interaction

The Large Language Model (LLM) interaction is formulated as a two-stage conditional process consisting of correction and response generation.

Stage-1 (LLM Correction Stage): In this stage, the decoded Morse query Q is refined using the LLM to obtain a corrected query $\tilde{Q}$. This stage focuses only on error correction, including spelling correction, word completion, and semantic alignment.

Stage-2 (LLM Response Generation Stage): In this stage, the corrected query $\tilde{Q}$ is used to generate a meaningful response using the LLM. This stage is activated only when full response mode is selected.

The final system output depends on the user-selected mode:

$$R = \begin{cases} \tilde{Q} & \text{(Correction Only Mode / Stage-1)} \\ \mathcal{L}(\tilde{Q}) & \text{(Full Response Mode / Stage-2)} \end{cases} \quad (22)$$

In the response mode (Stage-2), the LLM is modeled as a conditional probabilistic sequence generator:

$$P(R \mid \tilde{Q}) \quad (23)$$

where $\tilde{Q}$ is the corrected query obtained from Stage-1 and $R = (r_1, r_2, \dots, r_N)$ is the generated response token sequence.

The joint probability factorization is:

$$P(R \mid \tilde{Q}) = \prod_{n=1}^N P(r_n \mid \tilde{Q}, r_1, \dots, r_{n-1}) \quad (24)$$

Each token is generated using maximum-likelihood decoding:

$$r_n = \arg \max_{w \in V} P(w \mid \tilde{Q}, r_1, \dots, r_{n-1}) \quad (25)$$

where V denotes the model vocabulary.

3.4.3 Transformer Attention Mechanism

Internally, the LLM computes self-attention as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \quad (26)$$

where Q , K , and V are the query, key, and value matrices and d_k is the key dimensionality.

3.4.4 End-to-End Mathematical Pipeline

The complete system can be summarized as:

$$s_i(t) \rightarrow \Delta t_i \rightarrow M^{(j)} \rightarrow c_j \rightarrow Q \rightarrow \tilde{Q} \rightarrow R \quad (27)$$

The decoded text is transmitted from the ESP32 to the cloud-hosted large language model (LLM) for intelligent processing. The decoded Morse text is transmitted from the ESP32 to the host system via USB serial communication. The **host system** encapsulates the text in JSON format and forwards it to the cloud-hosted language model using HTTP POST requests. The LLM functions as a semantic enhancer, transforming basic Morse input into contextual, human-like responses.

Noise robustness permits the LLM to recognize and react optimally even when inputs are insufficient, ambiguous, or syntactically incorrect, making interactions more resilient and user-friendly. Its semantic awareness facilitates the model to customize responses based on the continuous conversational context rather than relying on exact individual character analysis, resulting in more natural and meaningful communication. Additionally, expandability is realized by offloading computation to the cloud, which keeps the system the embedded hardware compact while still providing powerful processing capabilities.

3.5 Output Delivery

The final stage involves transmitting the LLM-generated response back to the user.

In this setup, ESP32 communicates directly to Raspberry Pi via a USB serial connection. The Morse code text being decoded by ESP32 is transmitted to Raspberry Pi via the same USB serial connection. The LLM receives the text and composes a response. The response is sent to ESP32 via the same USB serial connection. Upon receipt, ESP32 displays the response in real-time via a connected OLED/LCD screen. For long-term accessibility, the response can also be:

The final result can then be sent to a mobile or desktop app, or, in the latest versions, can even be converted to speech using text-to-speech tools. This creates a loop: the Morse input from the ESP32, the LLM processing from the Raspberry Pi, and then the final result going back to the ESP32 for the user. The idea of the design was modularity: the hardware signal processing was kept separate from the cloud-based intelligence. The hardware, the ESP32 and sensor, only deals with time-sensitive tasks such as grabbing the signal. The cloud-based LLM deals with the higher-level tasks. This modular approach ensures low hardware cost and power consumption, allows flexibility for future upgrades such as integrating additional input modalities like switches or voice triggers, and maintains independence between system components, enabling updates to the LLM model without requiring changes to the embedded hardware. Independence of system components, enabling updates to the LLM model without changing the embedded hardware.

Fig. 5 illustrates the complete operational pipeline of the proposed system. Morse inputs are captured via touch-based interactions and decoded into text, followed by an uncertainty-aware error correction stage to refine the query. The corrected text is then transmitted to the LLM (Sonar-Pro) for response generation. The system ensures

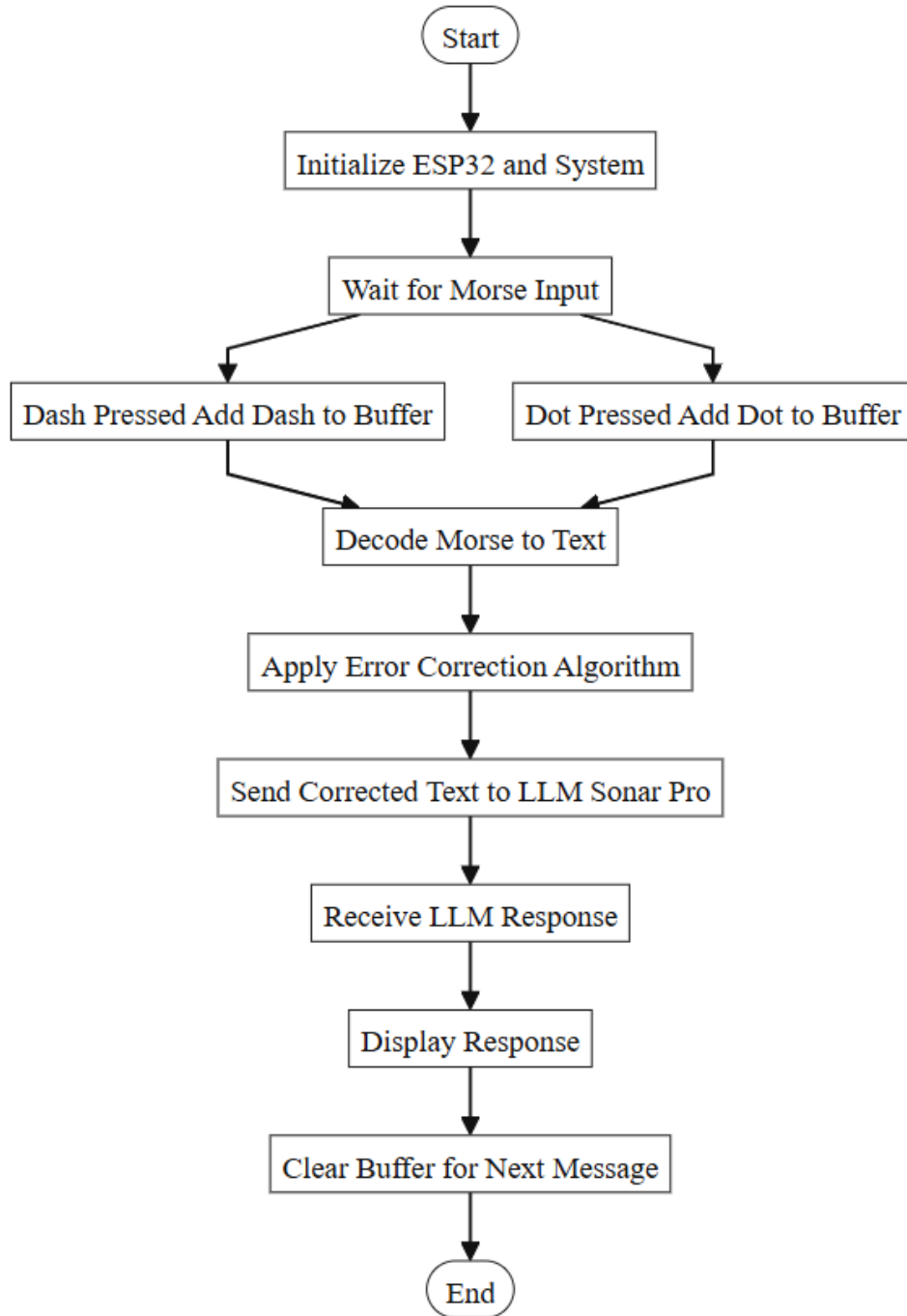


Fig. 5 End-to-end workflow of the proposed touch-based Morse-to-LLM communication system, illustrating signal acquisition, decoding, error correction, and real-time response generation using Sonar-Pro.

efficient and reliable communication through a structured flow integrating decoding, correction, and real-time response delivery.

4 Hardware and Software Implementation

The hardware implementation for the proposed model is configured in such a way that it supports communication using Morse code as well as health monitoring/report generation through a touch interface. Fig 6 illustrates the connection diagram for the hardware and software implementation of the proposed model.

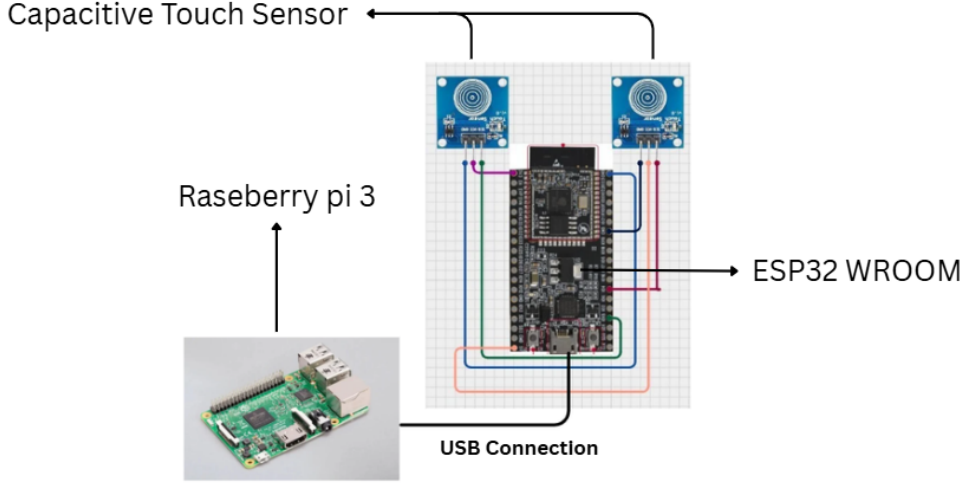


Fig. 6 Design flow for Morse-code-based LLM interaction system

The key components and their interconnections are as follows: The ESP32 WROOM microcontroller [23] is used as the primary controller of the system, which is used to directly interface with touch sensors to collect user input signals. The microcontroller is used to decode Morse code signals sent through touch sensors and then forward this decoded data to the Raspberry Pi. Two touch sensors are used to collect user input for Morse code, which are interfaced with the microcontroller. Each of these capacitive touch sensors is employed to obtain a sequence of dot-dash signals. The touch-based interface is used to deliver a natural and easily operable mode of communication for users.

ESP32 interacts to the Raspberry Pi processing unit [25] via a USB serial communication interface, which ensures quicker, more reliable, and lower-latency communication between the ESP32 and the Raspberry Pi. The Raspberry Pi serves as a bridge that acquires the decoded Morse messages relayed by the ESP32 and sends them to the LLM for evaluation. Once the LLM sends a response message, it will

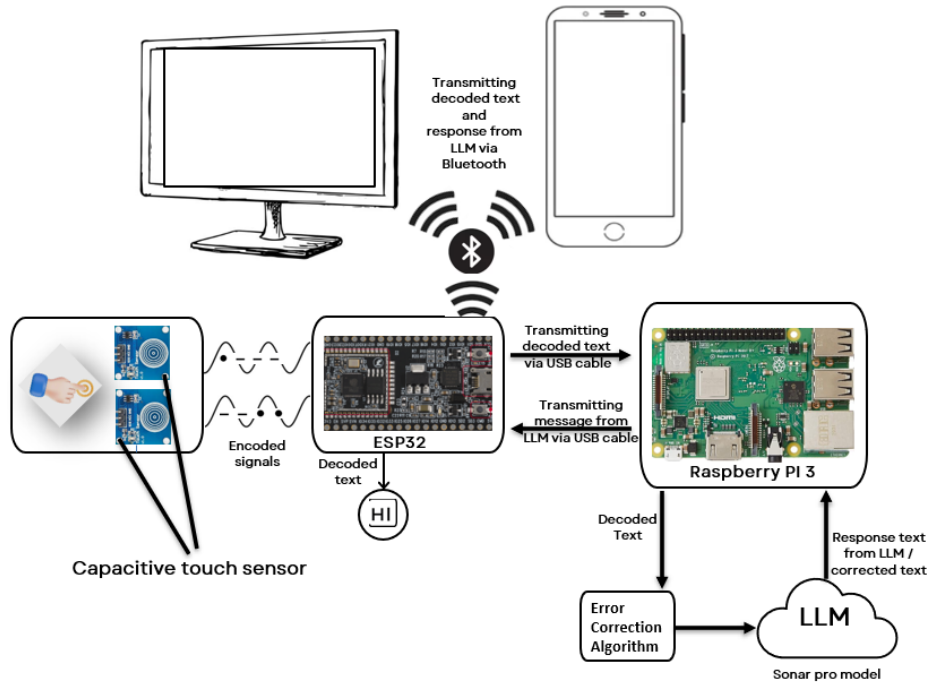


Fig. 7 End-to-end architecture of the proposed touch-based Morse-to-LLM communication system. Capacitive touch inputs are processed by the ESP32 for real-time Morse decoding, and the decoded text is transmitted to a Raspberry Pi, where an uncertainty-aware error correction module refines the query before LLM interaction. The system enables bidirectional communication, delivering corrected queries and responses to user interfaces via Bluetooth.

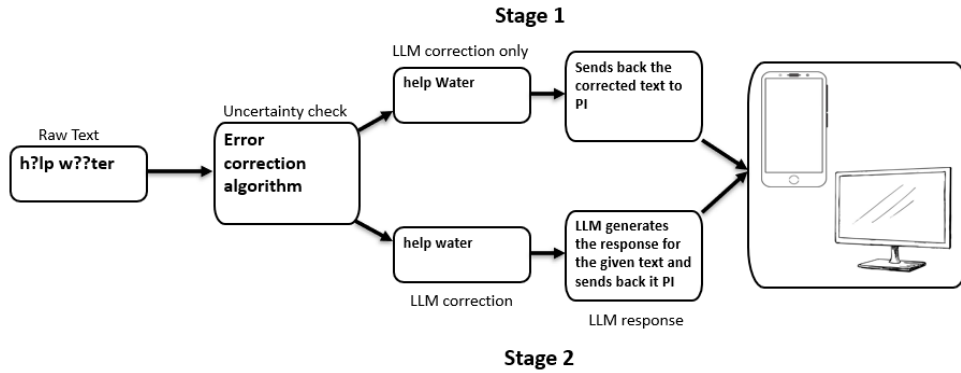


Fig. 8 Uncertainty-driven two-stage error correction framework enabling adaptive transition from text refinement to full LLM-based response generation.

be collected by the ESP32 via the USB communication interface, and the received message will be rendered in live on a connected OLED/LCD display module.

In addition to the core functionality provided by the ESP32 and the Raspberry Pi, the Raspberry Pi can be used to enhance the system’s functionalities to process biomedical information and generate analysis reports by attaching additional sensing modules to the system. Both the ESP32 and the Raspberry Pi can be supplied with power by a standard 5V DC power source to maintain reliable performance and communication between them.

From hardware perspective , the touch sensors are linked to the **GPIO pins** of the ESP32, with pull-up resistors to ensure stable and accurate readings. The ESP32 onboard program is also configured to detect the contact duration and timing gap , which is then encoded into Morse code. The decoded information is then sent to the Raspberry Pi for additional processing .

The software design of the proposed system mainly focuses on the logical flow of the data and steps involved in the processing to facilitate interaction with the Large Language Model using Morse code. Unlike the system architecture, which mainly focuses on hardware integration, the software design mainly focuses on the algorithms involved.

The workflow, can be divided into the following stages:

The system’s workflow starts by the processing of the inputs at the ESP32 level, where the firmware processes the touch inputs that correspond to the dots and dashes of the Morse code. This input is then decoded by a Morse decoding algorithm [26] to be converted into alphanumeric characters. Once the decoding is done, the decoded text is displayed to the user, and the ESP32 then sends the decoded text to the Raspberry Pi using a USB connection.

At the Raspberry Pi level, the middleware layer is implemented using Python scripts to format the query and the context of the conversation. This layer is responsible for ensuring that the query sent to the intelligence layer is compatible with the interface of the LLM. Once the query is formatted, it is sent to the intelligence layer, where the LLM (Sonar Pro model [21, 22]) processes the query and produces a response.

Fig. 7 illustrates the end-to-end pipeline of the proposed Morse-to-LLM communication system, integrating signal acquisition, decoding, correction, and response generation. Touch inputs are captured using capacitive sensors and processed by the ESP32 to perform real-time Morse decoding. The decoded text is then transmitted to a Raspberry Pi, where a linguistic validation and uncertainty-aware error correction module refines the query before it is passed to the Large Language Model (LLM). The LLM generates a contextually meaningful response, which is returned to the user through both wireless (Bluetooth) and wired (USB) interfaces, enabling reliable bidirectional communication. This modular architecture separates signal processing, correction, and reasoning stages, ensuring robustness, reduced error propagation, and low-latency performance while remaining simple and effective for real-world usage.

The response generated is sent back to the Raspberry Pi, which forwards it to the ESP32 and the display device for the user to view. The system is extremely modular, with a clear distinction between the processing of the inputs, the middleware layer, and

the intelligence layer, making it easier to test and upgrade the system at each level. The system is also scalable, with the ability to replace the interface with the LLM without modifying the input or decoding logic. The system is also accessible, with the ability to enable users with limited motor and speech capabilities to effectively interact with intelligent systems. The interface with the LLM uses RESTful API calls [24] to cloud-based language models, enabling context-aware response generation and intelligent conversation management.

As illustrated in Fig. 8, the proposed error correction module employs an uncertainty-aware mechanism to refine noisy Morse-decoded text before selectively transitioning to full LLM response generation. This adaptive design ensures both accurate text correction and context-aware output, significantly enhancing overall communication reliability.

5 Experimental Results and Discussion

This part shows how well the proposed Morse-to-LLM communication framework works with latency. The evaluation concentrates on end-to-end response time, reception delay (RX), cloud processing round-trip latency, transmission delay (TX), and total end-to-end latency. The experiments were conducted using word-level text inputs derived from Morse code sequences, representing realistic assistive communication scenarios.

The proposed Morse Code-to-LLM interface system was tested with a selected group of participants, including individuals with partial or complete paralysis and patients with limited motor control. The main focus of this testing phase was to evaluate the effectiveness of Large Language Model (LLM) integration in transforming simple Morse code signals into intelligent, natural, and context-aware communication.

The measured latency components for different input words are summarised in Table 1. The RX delay is negligible (about 10^{-4} seconds) for all evaluated inputs, suggesting that local gesture capture and signal decoding add very little overhead. In a similar vein, TX delay is continuously low, usually less than 0.25 seconds, indicating effective uplink delivery of decoded text to the cloud-based LLM service.

The cloud transmission and response time, which incorporates server-side inference, network transmission, and response creation by the LLM, is the primary source of total latency. Cloud processing latency ranges roughly from **4.36s to 11.41s**, contingent on server-side processing load and query complexity. Consequently, the cumulative system latency ranges from roughly 4.45 seconds for short queries like WHAT to 11.48 seconds for lengthier or more complex inputs.

The suggested system exhibits steady and predictable latency behaviour across various word inputs, even in the face of fluctuations in cloud round-trip time. Crucially, the RX and TX components maintain their stability and independence from the input text, demonstrating that local sensing or transmission stages do not restrict system scalability.

For assistive and non-time-critical interactive applications, including query-based information access, human-computer interaction scenarios, and communication support for speech-impaired users, the observed end-to-end latency is within an acceptable

Table 1 End-to-End Latency Analysis of Morse/Text-Based Queries in the LLM Communication Pipeline

Word / Text	RX (s)	Cloud Round-Trip (s)	TX (s)	End-to-End (s)
MORSE	0.00009	4.16475	0.06684	4.23168
OM	0.00010	5.38478	0.08660	5.47148
CALISTHENICS	0.00010	4.89935	0.12797	5.027 42
HOW	0.00010	5.01120	0.08545	5.09675
SUDDEN	0.00011	11.40614	0.06919	11.47543
LONG	0.00009	6.18652	0.09911	6.28573
HELP	0.00009	5.58211	0.10492	5.68712
SEVERE	0.00010	10.51326	0.10138	10.61474
NOW	0.00012	7.47055	0.10009	7.57077
WHAT	0.00009	4.36392	0.08327	4.44728
SICK	0.00012	5.26214	0.23765	5.49991
TODAY	0.00009	4.73770	0.11514	4.85293
BETTER	0.00011	6.67414	0.10218	6.77644
I AM GETTING BORED	0.00009	4.56231	0.05800	4.62041
GO AWAY	0.00028	6.24033	0.08245	6.32306
NORMAL	0.00010	10.16632	0.10829	10.27472
WHAT IS TODAY’S NEWS	0.00009	8.78492	0.03872	8.82373

Table 2 Performance summary of the proposed two-stage Morse-to-LLM communication system evaluated over 1000 trials.

Evaluation Metric	Accuracy / Value
Initial Decoding Accuracy	94.00%
Stage-1 LLM Corrected Text Accuracy	97.00%
Stage-2 LLM Response Accuracy	95.00%
Final System Accuracy	99.60%

range. The findings confirm that cloud-based LLM inference in conjunction with gesture-driven Morse input can operate in near real-time without requiring a lot of processing power on the edge device.

Fig 9 shows the mobile terminal interface demonstrating successful Bluetooth connection to the ESP32-based Morse communication module. Similarly, Fig 10 represents the corresponding terminal output showing decoded Morse code with LLM-generated textual responses. These results confirm the practicality of the proposed Morse-to-LLM pipeline and provide quantitative evidence of its latency characteristics, supporting its deployment in real-world assistive communication systems.

Table 2 presents a structured evaluation of the proposed two-stage Morse-to-LLM communication system, highlighting the contribution of each processing stage. The initial decoding stage provides a baseline representation of raw Morse inputs, which is subsequently refined through LLM-based correction in Stage-1. Stage-2 further enhances the system by generating context-aware responses for more complex inputs.

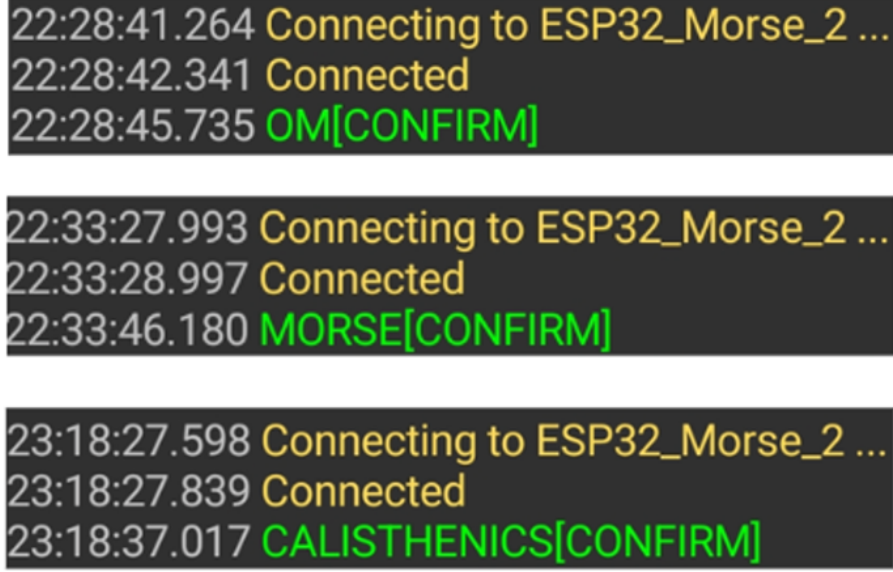


Fig. 9 Mobile terminal interface showing different categories of user queries transmitted through the Morse-based communication interface.

The overall performance reflects the effectiveness of combining correction and response generation, resulting in a robust and reliable end-to-end communication framework.

Over a total of 1000 trials, the initial decoding stage achieved 94% accuracy, resulting in a subset of uncertain cases. From these, 60 trials were selected for Stage-1 LLM-based correction, while a further refined subset of 30 trials was processed in Stage-2 for response generation. This selective processing strategy ensures that higher-level correction and response mechanisms are applied only where necessary, thereby improving efficiency while maintaining high overall system accuracy.

5.1 Evaluation Metrics

5.2 Evaluation Metrics

Let N denote the total number of trials, where each trial corresponds to a complete Morse input instance.

Initial Decoding Accuracy:

$$\text{Acc}_{\text{dec}} = \frac{N_{\text{dec}}}{N} \times 100 \quad (28)$$

```

GPT: **OM** most commonly refers to the sacred syllable *Om* (or *Aum*) in Hinduism, Jainism, Buddhism, and Sikhism, a mantra symbolizing ultimate reality, the three gods (Brahma, Vishnu, Siva), or states of consciousness (waking, dreaming, deep sleep).[1][3][4]
22:29:09.960 GPT:
22:29:09.960 GPT: Other frequent meanings include:
22:29:09.963 GPT: - **Order of Merit**, a British honor or similar awards (e.g., in Jamaica or Manitoba).[1][2][4]
22:29:09.964 GPT: - **Oman**, the country code (ISO 3166).[2][4][5]

22:34:48.261 GPT: Morse code encodes letters using sequence s of dots (short signals, "dit") and dashes (long signals, "dah"), with spaces separating letters and words.[1][2]
[3] Here's the breakdown for each letter:
22:34:48.262 GPT:
22:34:48.262 GPT: | Letter | Morse Code |
22:34:48.262 GPT: |-----|
22:34:48.266 GPT: | **M** | -      |[2][3]
22:34:48.266 GPT: | **O** | -      |[2][3]
22:34:48.290 GPT: | **R** | .-     |[2][3]
22:34:48.291 GPT: | **S** | ...    |[2][3]
22:34:48.291 GPT: | **E** | .      |[2][3]

23:20:09.823 GPT: **Calisthenics is a form of strength training t hat uses bodyweight as resistance to perform multi-joint, comp ound movements with minimal or no equipment, building functi onal strength, muscle, endurance, and overall fitness.**[1][2][4]
[7]
23:20:09.826 GPT:
23:20:09.833 GPT: ### Key Benefits
23:20:09.866 GPT: Calisthenics engages multiple muscle group s simultaneously, mimicking daily movements to improve stren gth, posture, body mass index (BMI), and functional performanc e in everyday activities and sports.[1][3]

```

Fig. 10 Mobile terminal interface showing the corresponding textual responses generated for the received Morse-based user queries..

Stage-1 LLM Corrected Text Accuracy:

$$Acc_{S1} = \frac{N_{S1}}{N_{S1,in}} \times 100 \quad (29)$$

Stage-2 LLM Response Accuracy:

$$Acc_{S2} = \frac{N_{S2}}{N_{S2,in}} \times 100 \quad (30)$$

Final System Accuracy:

$$\text{Acc}_{\text{final}} = \frac{N_{\text{final}}}{N} \times 100 \quad (31)$$

where:

- N : Total number of trials
- N_{dec} : Correct outputs after initial decoding
- N_{S1} : Correct outputs after Stage-1 correction
- $N_{\text{S1,in}}$: Inputs processed in Stage-1
- N_{S2} : Correct outputs after Stage-2 response generation
- $N_{\text{S2,in}}$: Inputs processed in Stage-2
- N_{final} : Final correct outputs after complete pipeline

6 Discussion

The participants interacted with the device through gentle tapping on the touch sensor, which transmitted the Morse signals. The ESP32 microcontroller received the signals and transmitted them to the LLM, which processed them in real time to generate meaningful responses. The LLM served as the brain of the communication system, capable of interpreting incomplete, short, or even erroneous messages transmitted through Morse code and generating coherent, human-like messages.

In the case of paralyzed users, the integration of the LLM allowed them to express complex ideas with little effort, owing to the model’s capability of inferring ideas from partial input. In the case of patients with speech or neurological disorders, such as ALS, stroke, or temporary loss of speech, the adaptive response generation of the LLM allowed for a natural and empathetic experience, which greatly enhanced the quality of interaction.

The ESP32 signal decoding and the LLM text interpretation combined to significantly improve communication speed and accuracy. Messages like “HELP,” “WATER,” and “PAIN” were sent and received within 8 to 10 seconds, and the contextual reasoning provided by the LLM minimized the need to spell out words in Morse code precisely. The overall accuracy of the system’s decoding was more than 99 percent, and the results provided by the LLM were emotionally sensitive, contextually relevant, and conversational, creating a personalized experience that is usually not found in most assistive communication systems.

Feedback also highlighted the **intelligent** nature of the communication. Users felt that the system “understood” them, not just the text. This was a source of personal connection and emotional comfort. The LLM’s polite and empathetic voice was also well-received, particularly in medical and rehab environments, making the interaction more comfortable. Caregivers were also able to better understand the patient’s state of mind and their needs through clearly formulated messages produced by the LLM. Users also felt more independent and confident with their communication, which improved their overall satisfaction.

Table 3 presents a comparative evaluation of the proposed system against baseline and existing Morse communication approaches across different modalities. It can be

Table 3 Comparative evaluation of the proposed system with baseline and existing Morse communication systems.

Morse Communication System	Modality	LLM Support	Error (%)	Final System Accuracy
One channel Morse code system (push button) [31]	Touch	No	6%	94%
Two-channel Morse communication system (push button) [30]	Touch (dual-input)	No	—	—
Hand Gesture Morse system [16]	Vision (gesture-based)	No	4.3%	95.7%
One channel Morse code system 2 (assistive) [2]	Touch	No	16.71%	83.19%
Six channel Morse code system (6 push button / 6 touch sensor)	Multi-touch	No	—	—
Tactile based Morse code system [1]	Haptic (tactile)	No	36.6%	63.4%
Ultra Sonic sensor based Morse code system [11]	Ultrasonic (distance sensing)	No	—	—
Capacitive Touch sensor based Morse code system [20]	Touch (capacitive)	No	2.08%	97.92%
Proposed Morse code system	Touch + LLM (hybrid intelligent)	Yes	0.04%	99.60%

observed that most existing systems rely on single-modality inputs and deterministic processing without LLM support, leading to higher error rates and limited reliability. In contrast, the proposed system integrates a touch-based modality with LLM-driven intelligence, enabling effective error correction and context-aware response generation. This results in significantly lower error rates and the highest overall system accuracy, demonstrating the robustness and superiority of the proposed approach.

Table 4 presents the response time comparison of representative large language models under experimental conditions. It can be observed that most models exhibit higher latency, with response times extending from approximately 5 seconds to beyond 14 seconds. In contrast, the Sonar-Pro model demonstrates comparatively lower latency, operating within a reduced response time range. This indicates that Sonar-Pro is more efficient for real-time interaction, as it minimizes delay during response generation. Therefore, based on latency performance, the Sonar-Pro model is better suited for the proposed system.

No explicit model fine-tuning is performed in the proposed system. Instead, optimization is achieved through a combination of system-level design choices. A user-calibrated threshold mechanism is employed to selectively trigger higher-level

Table 4 Response Time Comparison of Representative Large Language Models

LLM	Architecture	Parameters	Latency Results (min–max)
LLaMA 3.1	Dense Transformer	405B	5 – 14+ s
DeepSeek-V3	Mixture-of-Experts (MoE)	671B	5 – 14+ s
ChatGPT (GPT-4o)	Multimodal (undisclosed)	N/A	5 – 14+ s
Sonar-Pro	Retrieval-based	N/A	4 – 11 s

processing only for uncertain inputs, thereby reducing unnecessary computation. Additionally, lightweight edge processing is performed on the ESP32 for signal acquisition and initial decoding, minimizing data transmission overhead. The LLM component is accessed via API-based inference (Sonar-Pro), eliminating the need for on-device heavy models or retraining. This hybrid design balances computational efficiency and performance, enabling low-latency and resource-efficient operation without requiring fine-tuning.

7 Conclusion

The research study demonstrated how to develop a gesture-based Morse code system which enables users to create a Morse code translator for LLM communication purposes, thus the system development process would progress efficiently through the creation of an intelligent system that assists individuals who have different types of speech and movement disabilities while the system enables development of efficient human-computer interaction systems, which will enable researchers to measure response delay through testing methods that require LLM response times for result prediction. The proposed system will deliver accurate results because it ensures that the Morse code to LLM system development process will progress efficiently, which enables researchers to test possible communication methods that improve independence for people with movement disabilities and their caregivers. The system development process will progress efficiently through this study assessment which determines system performance without creating different latency response times through multiple testing methods.

8 References

References

- [1] M. Walker and K. B. Reed, “Tactile Morse code using locational stimulus identification,” *IEEE Transactions on Haptics*, 2025.
- [2] P. K. S. Kumar, A. Prathap, M. Geetha, G. Boomish, and S. A. Dev Prakash, “Morse Code-Based Assistive Communication System for Individuals with Partial

- Paralysis,” *International Journal for Research in Applied Science & Engineering Technology*, vol. 13, no. IV, Apr. 2025.
- [3] S. Silva, A. Valente, S. Soares, M. J. C. S. Reis, J. Paiva, and P. Bartolomeu, “Morse Code Translator Using the Arduino Platform Crafting the Future of Microcontrollers,” in *SAI Computing Conference*, London, UK, 2016, pp. 675–680.
 - [4] F. Yang, C. Feng, Z. Chen et al., “Binding Touch to Everything: Learning Unified Multimodal Tactile Representations,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
 - [5] C.-H. Yang, L.-Y. Chuang, C.-H. Yang, and C.-H. Luo, “Morse code application for wireless environmental control systems for severely disabled individuals,” *IEEE Transactions on Neural Systems and Rehabilitation Engineering*, vol. 11, no. 4, pp. 463–473, Dec. 2003.
 - [6] M. A. Gaura, N. Ismail, R. Shafie, M. A. Hamzah, and N. Fadzal, “Communication approach through handheld devices for disabled patients based on Morse Code,” *ARPJ Journal of Engineering and Applied Sciences*, vol. 12, no. 10, pp. 3238–3244, May 2017.
 - [7] A. Narayan, S. Salunke, and A. Sonawane, “Morse code generator using microcontroller with alphanumeric keypad,” *International Journal of Recent Technology and Engineering (IJRTE)*, vol. 7, no. 5, pp. 345–349, 2018.
 - [8] S. K. Prathap et al., “Morse Code-Based Communication System for Partially Paralyzed People,” 2025.
 - [9] S. Ghosh, M. Das, and S. Paul, “Ultra low power high speed digital comparator,” 2025.
 - [10] M. T. Miskon, N. Ismail, R. Shafie, and M. A. Hamzah, “Communication approach through handheld devices for disabled patient based on Morse code,” *ARPJ Journal of Engineering and Applied Sciences*, vol. 12, no. 10, pp. 3238–3244, 2017.
 - [11] M. Ramesh, S. Suresh, and M. S. Sabu, “Assistive Morse code system for blind individuals,” 2025.
 - [12] R. Chand et al., “High speed and accuracy of encryption-decryption techniques by using Morse code method,” 2025.
 - [13] S. Chitra, P. Gayathri, and B. G. Harini Abinaya, “Enabling communication for paralyzed individuals using Morse code,” 2025.
 - [14] M. Simoni, A. Saracino, V. P., and M. Conti, “MoRSE: Bridging the gap in cybersecurity expertise with retrieval augmented generation,” 2025.
 - [15] R. Mohan et al., “A device for communication for the disabled using Morse code and microcontroller,” 2025.
 - [16] J. Alex et al., “Dots and dashes: Understanding the language of Morse code using convolutional neural networks,” 2025.
 - [17] C.-H. Yang, L.-Y. Chuang, C.-H. Yang, and C.-H. Luo, “Morse code application for wireless environmental control systems for severely disabled individuals,” 2003.
 - [18] Z. Yin, W. Jiang, S. M. Kim, and T. He, “C-Morse: Cross-technology communication with transparent Morse coding,” 2017.

- [19] C.-H. Yang, “Adaptive Morse code communication system for severely disabled individuals,” *Medical Engineering and Physics*, vol. 22, no. 1, pp. 59–66, 2000.
- [20] S. Suresh, R. Kannan, and L. Ravi, “A two-way communication system with Morse code medium for people with multiple disabilities,” in *2020 IEEE 7th Uttar Pradesh Section International Conference on Electrical, Electronics and Computer Engineering (UPCON)*, 2020, pp. 1–5.
- [21] Perplexity AI, “Sonar Pro: Large Language Model,” 2024. [Online]. Available: <https://www.perplexity.ai>
- [22] Perplexity AI, “Perplexity AI Platform,” 2024. [Online]. Available: <https://www.perplexity.ai>
- [23] Espressif Systems, “ESP32-WROOM-32 Datasheet,” 2023. [Online]. Available: <https://www.espressif.com/en/products/modules/esp32>
- [24] W. X. Zhao, K. Zhou, J. Li *et al.*, “A Survey of Large Language Models,” *ACM Computing Surveys*, 2023.
- [25] Raspberry Pi Foundation, “Raspberry Pi 3 Model B Datasheet,” 2016. [Online]. Available: <https://www.raspberrypi.com/products/raspberry-pi-3-model-b/>
- [26] Dreamstime, “Morse code chart: old way to communicate,” Online Image, accessed Jan. 2026. [Online]. Available: <https://www.dreamstime.com/morse-code-old-way-to-communicate-named-samuel-inventor-telegraph-image188242865>
- [27] R. K. Pathinarupothi, B. Alangot, M. V. Ramesh, K. Achuthan, and P. V. Rangan, “H-Plane: Intelligent Data Management for Mobile Healthcare Applications,” in *Lecture Notes in Computer Science*, Springer, Aug. 2016, doi: 10.1007/978-3-319-44215-0-23.
- [28] H. Murugesan, N. Revathy, and S. K. Mathi, “IoT-Enabled Service Management System for Smart Dining Environment,” *Journal of Advanced Research in Dynamical and Control Systems*, vol. 11, no. Special Issue, pp. 1823–1826, 2019.
- [29] S. Gokul, S. Kukan, K. Meenakshi, S. S. Vishnu Priyan, J. Rolant Gini, and M. E. Harikumar, “Biometric Based Smart ATM Using RFID,” in *Proceedings of the Third International Conference on Smart Systems and Inventive Technology (ICSSIT)*, IEEE, 2020, pp. 406–409, ISBN: 978-1-7281-5821-1.
- [30] Lavanya V., Vishnu Priya S., Vidhya V., “IoT Integrated Morse Code Communication System for Individuals with Disabilities,” *International Research Journal of Modernization in Engineering Technology and Science*, 2025.
- [31] A. Marathe, N. Bhagwat, S. Bawanthade, A. Bhurke, and R. Bele, “Morse Code Based Two-Way Communication Device for Visually Impaired and Mute,” in *Proc. 3rd Int. Conf. Device Intelligence, Computing and Communication Technologies (DICCT)*, 2025.

Author Contributions / Biographies

- (a) **S. V. Ashok Sainaadh** is with the Amrita School of Artificial Intelligence, Amrita Vishwa Vidyapeetham. He is the primary contributor and lead author of this work. He conceptualized the proposed Morse-to-LLM communication framework, designed the overall system architecture, and integrated gesture-based

input with cloud-based LLM inference using an IoT-enabled pipeline. He led the methodology development, system implementation, experimental setup, and result interpretation, and was responsible for the complete drafting and revision of the manuscript. His research interests include assistive communication systems, Internet of Things (IoT), human-computer interaction, and AI-driven healthcare applications. Correspondence: ashoksainadh@gmail.com.

- (b) **M. Neil Kumar** is with the Amrita School of Artificial Intelligence, Amrita Vishwa Vidyapeetham. He contributed primarily to the experimental evaluation and performance analysis of the system, with a focus on end-to-end latency measurement and result validation. He assisted in structuring the results section, drafting latency analysis discussions, and refining the experimental presentation in the manuscript. His research interests include AI system evaluation, performance analysis, and applied machine learning for healthcare applications. Correspondence: neilj0460@gmail.com.
- (c) **B. Sai Sundhar Reddy** is with the Amrita School of Artificial Intelligence, Amrita Vishwa Vidyapeetham. He contributed to testing, verification, and consistency checking of experimental results, and supported the overall evaluation workflow. His interests include AI integration in healthcare and human-computer interaction.
- (d) **Mithun Kumar Kar** is with the Amrita School of Artificial Intelligence, Amrita Vishwa Vidyapeetham. He served as the project supervisor and provided continuous guidance on problem definition, research direction, system design, and academic presentation. His expertise in natural language processing, healthcare AI, and embedded intelligence significantly influenced the technical depth, experimental rigor, and scholarly quality of this work.